

SOCKET:

Q1. Define Socket.

A socket is an endpoint for communication between two machines. In Java, the Socket class is used to establish a connection between a client and a server using the TCP/IP protocol.

- It allows bidirectional communication between client and server.
- A socket is bound to a port number so that the TCP layer can identify the application that data is destined to.

Types of Sockets in Java:

- Socket – used for client-side communication.
- ServerSocket – used for server-side communication.

Q2. Write a program to create a server socket. Explain the procedure for connecting and creating the server socket.

```
import java.io.*;
```

```
import java.net.*;
```

```
public class SimpleServer {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Step 1: Create a ServerSocket listening on port 5000
```

```
            ServerSocket serverSocket = new ServerSocket(5000);
```

```
            System.out.println("Server is listening on port 5000...");
```

```
// Step 2: Wait for the client to connect

Socket socket = serverSocket.accept(); // blocking call

System.out.println("Client connected!");


// Step 3: Get input and output streams

InputStream input = socket.getInputStream();

BufferedReader reader = new BufferedReader(new InputStreamReader(input));


OutputStream output = socket.getOutputStream();

PrintWriter writer = new PrintWriter(output, true);


// Step 4: Read data from client and send response

String clientMessage = reader.readLine();

System.out.println("Received from client: " + clientMessage);

writer.println("Hello from server!");


// Step 5: Close connections

socket.close();

serverSocket.close();

} catch (IOException e) {
```

```

        e.printStackTrace();
    }
}
}

```

What is the procedure for constructing and connecting Server sockets?

💡 Step-by-Step Explanation of the Program:

Step	Code	Explanation	📄
1. Create a ServerSocket	<pre>ServerSocket serverSocket = new ServerSocket(5000);</pre>	Opens a server socket on port <code>5000</code> . It will listen for client connection requests.	
2. Accept a client	<pre>Socket socket = serverSocket.accept();</pre>	Waits (blocks) until a client connects. When a client connects, it returns a <code>Socket</code> object representing the connection.	
3. Input stream	<pre>InputStream input = socket.getInputStream(); BufferedReader reader = new BufferedReader(new InputStreamReader(input));</pre>	Reads data sent from the client.	
4. Output stream	<pre>OutputStream output = socket.getOutputStream(); PrintWriter writer = new PrintWriter(output, true);</pre>	Sends data back to the client.	
5. Read/Write	<pre>reader.readLine() and writer.println(...)</pre>	Read message from the client and send a response to the client.	
6. Close sockets	<pre>socket.close(); serverSocket.close();</pre>	Close the connection and release resources.	

Q3. Write a program to create a client socket. Explain the procedure for connecting and creating the client socket.

✅ Client Socket Program (TCP Example)

```

import java.io.*;

import java.net.*;

```

```
public class SimpleClient {  
  
    public static void main(String[] args) {  
  
        try {  
  
            // Step 1: Connect to the server at localhost and port 5000  
  
            Socket socket = new Socket("localhost", 5000);  
  
            System.out.println("Connected to the server!");  
  
  
            // Step 2: Get output stream to send data to server  
  
            OutputStream output = socket.getOutputStream();  
  
            PrintWriter writer = new PrintWriter(output, true);  
  
            writer.println("Hello Server, this is Client!");  
  
  
            // Step 3: Get input stream to receive data from server  
  
            InputStream input = socket.getInputStream();  
  
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));  
  
            String serverMessage = reader.readLine();  
  
            System.out.println("Server says: " + serverMessage);  
  
  
            // Step 4: Close the socket  
  
            socket.close();  
  
        } catch (IOException e) {
```

```

        e.printStackTrace();
    }
}
}

```

💡 Step-by-Step Explanation of the Program

Step	Code	Explanation	
1. Connect to Server	<code>Socket socket = new Socket("localhost", 5000);</code>	Creates a client socket and connects to the server running on <code>localhost</code> at port <code>5000</code> .	
2. Output stream	<code>OutputStream output = socket.getOutputStream(); PrintWriter writer = new PrintWriter(output, true);</code>	Sends data to the server using <code>PrintWriter</code> .	
3. Input stream	<code>InputStream input = socket.getInputStream(); BufferedReader reader = new BufferedReader(new InputStreamReader(input));</code>	Reads the response from the server.	
4. Close socket	<code>socket.close();</code>	Closes the connection and releases the resources.	

✅ What is a Daytime Service?

A **daytime service** is a simple server program that sends the **current date and time** to any client that connects to it. The client doesn't send anything; it just receives the date and time from the server.

✅ Server-Side Program (Daytime Server using Socket in Java)

```
import java.io.*;
```

```
import java.net.*;

import java.util.Date;


public class DaytimeServer {

    public static void main(String[] args) {

        try {

            // Step 1: Create a ServerSocket on port 6000

            ServerSocket serverSocket = new ServerSocket(6000);

            System.out.println("Daytime server is running on port 6000...");


            while (true) {

                // Step 2: Accept client connection

                Socket socket = serverSocket.accept();

                System.out.println("Client connected.");


                // Step 3: Send current date and time to client

                PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

                String currentDateTime = new Date().toString(); // Get system date and time

                out.println("Current Date & Time: " + currentDateTime);


                // Step 4: Close the client connection

                socket.close();
```

```
    }

    } catch (IOException e) {

        e.printStackTrace();

    }

}

}
```

```
import java.io.*;
```

```
import java.net.*;
```

```
public class DaytimeClient {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Step 1: Connect to the server on localhost and port 6000
```

```
            Socket socket = new Socket("localhost", 6000);
```

```
            System.out.println("Connected to the server.");
```

```
            // Step 2: Read the date and time from the server
```

```
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
```

```
            String serverResponse = in.readLine();
```

```
System.out.println("Server says: " + serverResponse);
```

```
// Step 3: Close the connection
```

```
socket.close();
```

```
} catch (IOException e) {
```

```
    e.printStackTrace();
```

```
}
```

```
}
```

```
}
```

Explanation for Exam:

- `ServerSocket serverSocket = new ServerSocket(6000);`
Creates a server socket that listens on port 6000.
- `Socket socket = serverSocket.accept();`
Waits for a client to connect. (Blocking call)
- `new Date().toString();`
Gets the current system date and time.
- `PrintWriter out = new PrintWriter(socket.getOutputStream(), true);`
Creates an output stream to send data to the client.
- `out.println(...);`
Sends the current date/time as a message.
- `socket.close();`
Closes connection after sending the response.

Sample Output (Server Console):

arduino

 Copy  Edit

```
Daytime server is running on port 6000...  
Client connected.  
Client connected.  
Client connected.
```



 **Q: Write a program to display the socket information [address, port, local address, local port].**

Explanation:

To display socket information, we create a **client socket**, connect it to a server (or a known address), and then use built-in methods to fetch:

- `getInetAddress()` → Remote IP address
- `getPort()` → Remote port

- `getLocalAddress()` → Local IP address
 - `getLocalPort()` → Local port
-

Program to Display Socket Information

```
import java.io.*;
```

```
import java.net.*;
```

```
public class SocketInfo {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Create a socket connecting to google.com on port 80 (HTTP)
```

```
            Socket socket = new Socket("google.com", 80);
```

```
            // Display socket information
```

```
            System.out.println("Remote Address    : " + socket.getInetAddress());
```

```
            System.out.println("Remote Port      : " + socket.getPort());
```

```
            System.out.println("Local Address    : " + socket.getLocalAddress());
```

```
            System.out.println("Local Port      : " + socket.getLocalPort());
```

```
            System.out.println("Is Connected?    : " + socket.isConnected());
```

```
            System.out.println("Is Closed?       : " + socket.isClosed()); OTHER INFO OPTIONS AVAILABLE
```

```
IN SYLLABUS
```

```

        // Close the socket

        socket.close();

    } catch (IOException e) {

        e.printStackTrace();

    }
}
}
}

```

WHAT EACH ONE MEANS:

Method / Property	What It Means	
<code>getInetAddress()</code>	IP address of the server you're connected to (remote host)	
<code>getPort()</code>	The port number on the server (remote port, e.g., 80)	
<code>getLocalAddress()</code>	Your own machine's IP address (client side)	
<code>getLocalPort()</code>	The port your machine used for this connection (assigned randomly by OS)	
<code>isConnected()</code>	Returns <code>true</code> if socket was successfully connected , even if later closed	
<code>isClosed()</code>	Returns <code>true</code> if the socket is closed	
<code>toString()</code>	Returns a full line of socket info in this format: <code>addr=google.com/142.250.196.142,port=80,localport=58622</code>	

Sample Output (may vary depending on network):

yaml

 Copy  Edit

```
Remote Address    : google.com/142.250.195.14
Remote Port       : 80
Local Address     : /192.168.1.10
Local Port        : 54321
```

Explanation of Methods:

Method	Purpose
<code>getInetAddress()</code>	Returns the remote host's IP address
<code>getPort()</code>	Returns the remote port number
<code>getLocalAddress()</code>	Returns the IP address of the client (your machine)
<code>getLocalPort()</code>	Returns the port your socket is using locally

What Are Server Socket Options?

Server socket options are configuration parameters used to customize the behavior of a server socket. These options control various aspects such as timeouts, buffering, address reuse, connection keep-alive, etc.

They are typically set using:

java

CopyEdit

`Socket.setOption()` // for regular Socket

`ServerSocket.setOption()` // for server sockets

Common Socket Options in Your Syllabus (with Example)

Option	Meaning	Example Java Usage
TCP_NODELAY	Disables Nagle's algorithm; sends packets immediately without waiting.	<code>socket.setTcpNoDelay(true);</code>
SO_LINGER	Controls how long to wait before closing a socket with data in buffer.	<code>socket.setSoLinger(true, 10);</code>
SO_TIMEOUT	Sets timeout for blocking socket operations (like read).	<code>socket.setSoTimeout(5000);</code>
SO_RCVBUF	Sets the size of the receive buffer.	<code>socket.setReceiveBufferSize(8192);</code>
SO_SNDBUF	Sets the size of the send buffer.	<code>socket.setSendBufferSize(8192);</code>
SO_KEEPALIVE	Sends keep-alive packets to detect dead connections.	<code>socket.setKeepAlive(true);</code>
OOBINLINE	Receives urgent data inline (if any).	<code>socket.setOOBInline(true);</code>
SO_REUSEADDR	Allows the reuse of a local address/port even if in TIME_WAIT state.	<code>serverSocket.setReuseAddress(true);</code>
IP_TOS	Sets the Type of Service field in the IP header (QoS, priority, etc.).	<code>socket.setTrafficClass(0x10);</code>

TIME_WAIT is a state in the TCP connection lifecycle where the socket remains open for a specific period (typically $2 * \text{Maximum Segment Lifetime}$ or $2 * \text{MSL}$, which is often around 2 minutes), even after the connection has been closed.

PROGRAM FIRST THEN SEE THIS:

Why are we using `bind()` here instead of passing the port directly?

You're absolutely right that usually we do this:

```
java
ServerSocket serverSocket = new ServerSocket(5000);
```

This is a **shortcut** constructor that:

1. **Creates** the socket.
2. **Binds** it to port `5000`.

But when you write:

```
java
ServerSocket serverSocket = new ServerSocket();
serverSocket.setReuseAddress(true);
serverSocket.setSoTimeout(10000);
serverSocket.bind(new InetSocketAddress(5000));
```

You're doing **each step manually**, because:

- You want to set **options like** `SO_TIMEOUT` **or** `SO_REUSEADDR` **before** binding the socket.
- Once a socket is **bound**, **some options can no longer be changed** (like reuse address).



Java Program Demonstrating All Server Socket Options.

```
import java.io.*;
```

```
import java.net.*;
```

```
public class SocketOptionsDemo {

    public static void main(String[] args) throws IOException {

        // Create server socket
```

```
ServerSocket serverSocket = new ServerSocket();

// 1. SO_REUSEADDR – reuse port
serverSocket.setReuseAddress(true);

// 2. SO_TIMEOUT – timeout for accept()
serverSocket.setSoTimeout(10000); // 10 seconds

// Bind to port
serverSocket.bind(new InetSocketAddress(5000));

System.out.println("Server started on port 5000. Waiting for client...");

try {

    // Accept client connection

    Socket socket = serverSocket.accept();

    System.out.println("Client connected!");

    // 3. TCP_NODELAY – disables Nagle's algorithm
    socket.setTcpNoDelay(true);

    // 4. SO_LINGER – linger on close if unsent data
    socket.setSoLinger(true, 5); // linger 5 sec before close
```

// 5. SO_TIMEOUT – read timeout on socket

socket.setSoTimeout(5000); // 5 sec read timeout

// 6. SO_RCVBUF – receive buffer size

socket.setReceiveBufferSize(16 * 1024); // 16 KB

// 7. SO_SNDBUF – send buffer size

socket.setSendBufferSize(16 * 1024); // 16 KB

// 8. SO_KEEPALIVE – keep-alive messages

socket.setKeepAlive(true);

// 9. OOBINLINE – urgent data received inline

socket.setOOBInline(true);

// 10. IP_TOS – type of service (QoS)

socket.setTrafficClass(0x10); // IPTOS_LOWDELAY

// Display current settings

System.out.println("TCP_NODELAY: " + socket.getTcpNoDelay());

System.out.println("SO_LINGER: " + socket.getSoLinger());

```
System.out.println("SO_TIMEOUT: " + socket.getSoTimeout());

System.out.println("SO_RCVBUF: " + socket.getReceiveBufferSize());

System.out.println("SO_SNDBUF: " + socket.getSendBufferSize());

System.out.println("SO_KEEPALIVE: " + socket.getKeepAlive());

System.out.println("OOBINLINE: " + socket.getOOBInline());

System.out.println("IP_TOS: " + socket.getTrafficClass());


// Clean up

socket.close();

} catch (SocketTimeoutException e) {

    System.out.println("No client connected within timeout.");

}

serverSocket.close();

}

}
```

What you do if you don't want to set options but still want to bind later:

```
java                                                                    Copy Edit

ServerSocket serverSocket = new ServerSocket(); // Step 1: Create unbound ServerSocket

// You skip any option setting like setReuseAddress or setSoTimeout here

serverSocket.bind(new InetSocketAddress(5000)); // Step 2: Bind later manually to port 5000
```

CLIENT SOCKET OPTS

Common Configuration Options:

Option	Purpose & Usage	
<code>setSoTimeout(int timeoutMillis)</code>	Sets the maximum time in milliseconds the socket will block while waiting for data on <code>read()</code> calls. If the timeout expires, a <code>SocketTimeoutException</code> is thrown. Useful to avoid infinite blocking.	
<code>setTcpNoDelay(boolean on)</code>	Enables/disables Nagle's algorithm. When <code>true</code> , data is sent immediately without delay, improving real-time communication but possibly increasing network packets. Default is <code>false</code> .	
<code>setReuseAddress(boolean on)</code>	Allows the socket to bind to a port that is in the <code>TIME_WAIT</code> state after a previous connection closed. Useful when restarting clients frequently.	
<code>setKeepAlive(boolean on)</code>	Enables TCP keepalive probes to check if the connection is still alive, helping detect dead peers.	
<code>setReceiveBufferSize(int size)</code>	Sets the size (in bytes) of the socket's receive buffer, which can impact throughput and performance.	
<code>setSendBufferSize(int size)</code>	Sets the size (in bytes) of the socket's send buffer.	
<code>setSoLinger(boolean on, int seconds)</code>	Controls the socket close behavior: if enabled, the socket tries to send remaining data for the specified seconds before closing.	
<code>bind(SocketAddress localAddr)</code>	Binds the socket to a specific local IP address and port before connecting. Useful if the client machine has multiple network interfaces.	

SAME TEI OPTIONSS KO KURAKO CLIENT KO DEKHA OR SERVER KO DEKHA CHUTAI VANDA DEKHAUNE PRIORAM: (BECAUSEE CONNECT JUN MA HUNCAH TETA LAGNE HO NITA UYO OPTIONBS HARU SO CLICENT KO SIDEI SOCKETY MA LAGYO BUT SERVER MA CLIENT SOCKET MA LAGYO BECAUSE CLIENT ACCEPT GARCHA ANI CONNECTION VAKO SOCKET IS CLIENT SOCKET SO TESMA LAGYO)

PROGRAM : SERVER SIDE:

```
import java.io.*;
```

```
import java.net.*;

public class ConfiguredServer {

    public static void main(String[] args) {

        try {

            // Step 1: Create a ServerSocket without binding yet

            ServerSocket serverSocket = new ServerSocket();

            // Step 2: Set configuration options BEFORE binding

            serverSocket.setReuseAddress(true);    // Allow reuse of the port

            serverSocket.setSoTimeout(10000);      // Timeout for accept() = 10 seconds

            // Step 3: Bind to port 5000

            serverSocket.bind(new InetSocketAddress(5000));

            System.out.println("Server started on port 5000. Waiting for client...");

            // Step 4: Accept incoming client connection (waits up to 10 seconds)

            Socket clientSocket = serverSocket.accept(); // Blocking call

            System.out.println("Client connected!");

            // Step 5: Configure accepted socket (connected socket)

            clientSocket.setKeepAlive(true);      // Enable TCP keepalive
```

```

clientSocket.setTcpNoDelay(true);    // Disable Nagle's algorithm

clientSocket.setSoLinger(true, 5);    // Wait 5 seconds before force close

clientSocket.setReceiveBufferSize(8192);    // Optional tuning

clientSocket.setSendBufferSize(8192);


// Communication

BufferedReader    reader    =    new    BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));

PrintWriter writer = new PrintWriter(clientSocket.getOutputStream(), true);


String msg = reader.readLine();

System.out.println("Client says: " + msg);

writer.println("Hello from server!");


// Cleanup

clientSocket.close();

serverSocket.close();

} catch (SocketTimeoutException ste) {

    System.out.println("No client connected within timeout.");

} catch (IOException e) {

    e.printStackTrace();

}

```

```
}  
  
}
```

PROGRAM : CLIENT SIDE:

```
import java.io.*;
```

```
import java.net.*;
```

```
public class ConfiguredClient {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Step 1: Create a socket object without connecting yet
```

```
            Socket socket = new Socket();
```

```
            // Step 2: Set configuration options BEFORE connecting
```

```
            socket.setReuseAddress(true);           // Allow reuse of local port
```

```
            socket.setSoTimeout(5000);             // Read timeout of 5 seconds
```

```
            socket.setTcpNoDelay(true);            // Send data immediately
```

```
            socket.setKeepAlive(true);             // Keep the connection alive
```

```
            socket.setReceiveBufferSize(8192);     // Optional
```

```
            socket.setSendBufferSize(8192);
```

```
            // Step 3: Connect to server
```

```
socket.connect(new InetSocketAddress("localhost", 5000));

System.out.println("Connected to server!");


// Communication

PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);

BufferedReader reader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));


writer.println("Hi Server, this is the client!");

String reply = reader.readLine();

System.out.println("Server says: " + reply);


// Cleanup

socket.close();

} catch (IOException e) {

    e.printStackTrace();

}

}

}
```

Multithreaded TCP Daytime Server

```
import java.io.*;

import java.net.*;

import java.util.Date;


public class MultiThreadedDaytimeServer {

    public static void main(String[] args) {

        int port = 5000;


        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Multithreaded Daytime Server is running on port " + port);


            while (true) {

                // Accept client connection

                Socket clientSocket = serverSocket.accept();

                System.out.println("Client connected from: " + clientSocket.getInetAddress());


                // Create a new thread to handle the client

                ClientHandler handler = new ClientHandler(clientSocket);

                Thread thread = new Thread(handler);

                thread.start();

            }

        }
```

```
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}  
}
```

// Runnable class to handle client requests

```
class ClientHandler implements Runnable {
```

```
    private Socket socket;
```

```
    public ClientHandler(Socket socket) {
```

```
        this.socket = socket;
```

```
    }
```

```
@Override
```

```
public void run() {
```

```
    try {
```

```
        // Send current date and time
```

```
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```
        String currentTime = new Date().toString();
```

```
        out.println("Current Date and Time: " + currentTime);
```

```

        System.out.println("Handled client: " + socket.getInetAddress());

        socket.close();

    } catch (IOException e) {

        e.printStackTrace();

    }

}

}

```

Multithreaded TCP PrimeNumber Server

```

import java.io.*;

import java.net.*;

public class PrimeServer {

    public static void main(String[] args) throws IOException {

        ServerSocket serverSocket = new ServerSocket(5000);

        System.out.println("  Prime Server started on port 5000...");

        while (true) {

            Socket clientSocket = serverSocket.accept();

            System.out.println("Client connected from: " + clientSocket.getInetAddress() + ":" +
            clientSocket.getPort());

```

```
        // Explicit thread creation for clarity

        ClientHandler handler = new ClientHandler(clientSocket);

        Thread thread = new Thread(handler);

        thread.start();

    }

}
```

```
class ClientHandler implements Runnable {
```

```
    private Socket socket;
```

```
    public ClientHandler(Socket socket) {
```

```
        this.socket = socket;
```

```
    }
```

```
    @Override
```

```
    public void run() {
```

```
        try (
```

```
            DataInputStream in = new DataInputStream(socket.getInputStream());
```

```
            DataOutputStream out = new DataOutputStream(socket.getOutputStream());
```

```
        ){
```

```
int number = in.readInt();
```

```
String result = isPrime(number) ? "Prime" : "Composite";
```

```
out.writeUTF(result);
```

```
System.out.println("Handled client " + socket.getInetAddress() + ":" + socket.getPort()
```

```
+
```

```
    "-> Number: " + number + " => " + result);
```

```
    } catch (IOException e) {
```

```
        e.printStackTrace();
```

```
    } finally {
```

```
        try {
```

```
            socket.close();
```

```
        } catch (IOException ex) {
```

```
            ex.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```

```
private boolean isPrime(int n) {
```

```
    if (n <= 1) return false;
```

```
    if (n == 2) return true;
```

```
    if (n % 2 == 0) return false;
```

```
for (int i = 3; i <= Math.sqrt(n); i += 2) {  
    if (n % i == 0) return false;  
}  
return true;  
}  
}
```

What is Java Secure Sockets Extension (JSSE)?

Java Secure Sockets Extension (JSSE) is a set of Java APIs that enable secure **Internet communications**. It provides:

- **Secure socket communication (SSL/TLS)** between client and server.
 - Support for both **client-side and server-side authentication**.
 - Strong encryption algorithms to ensure **data integrity, confidentiality, and authentication**.
-

Key Components of JSSE:

- **SSLSocket / SSLServerSocket**: Secure versions of Socket / ServerSocket.
 - **SSLContext**: Creates secure socket factories with specific protocol settings.
 - **KeyStore and TrustStore**: Manage private keys and certificates.
 - **KeyManager / TrustManager**: Control authentication and trust decisions.
-

 For tufoshss.edu.np (assuming it's using HTTPS):

You can create a secure client socket using JSSE to connect to a live HTTPS server like <https://tufoshss.edu.np>.

💡 This program works only **if the site supports TLS/SSL and allows socket-level communication.**

✅ Example: Secure Client Socket with JSSE to Connect to tufoshss.edu.np

```
import javax.net.ssl.*;

import java.io.*;

import java.security.cert.X509Certificate;

public class SecureClientTufoshss {

    public static void main(String[] args) {

        try {

            // Trust all certificates (for demo purposes only!)

            TrustManager[] trustAll = new TrustManager[] {

                new X509TrustManager() {

                    public X509Certificate[] getAcceptedIssuers() { return null; }

                    public void checkClientTrusted(X509Certificate[] certs, String authType) {}

                    public void checkServerTrusted(X509Certificate[] certs, String authType) {}

                }

            };

        }

    };

}
```

```

SSLContext context = SSLContext.getInstance("TLS");

context.init(null, trustAll, new java.security.SecureRandom());

SSLSocketFactory factory = context.getSocketFactory();


// Connect to tufoshss.edu.np at port 443 (HTTPS port)

try (SSLSocket socket = (SSLSocket) factory.createSocket("tufoshss.edu.np", 443)) {

    socket.startHandshake();


    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

    BufferedReader      in      =      new      BufferedReader(new
InputStreamReader(socket.getInputStream()));


    // Send HTTP GET request

    out.println("GET / HTTP/1.1");

    out.println("Host: tufoshss.edu.np");

    out.println("Connection: close");

    out.println(); // blank line to end the request


    String line;

    while ((line = in.readLine()) != null) {

        System.out.println(line);

    }
}

```

```

    }

    } catch (Exception e) {

        e.printStackTrace();

    }

}

}

```

UDP SOCKETS:

Hey! Yeah, client and server socket programs are different roles but both use sockets to communicate. In **UDP (User Datagram Protocol)**, communication is connectionless and messages (datagrams) are sent without establishing a connection, unlike TCP.

Here's a simple example of UDP client and server programs in Java:

UDP Server (listens for messages)

```

import java.net.DatagramPacket;

import java.net.DatagramSocket;

public class UDPServer {

    public static void main(String[] args) throws Exception {

        DatagramSocket serverSocket = new DatagramSocket(9876); // Bind server socket to
port 9876

        byte[] receiveBuffer = new byte[1024];

```

```
System.out.println("UDP Server is running...");

while (true) {

    DatagramPacket    receivePacket    =    new    DatagramPacket(receiveBuffer,
receiveBuffer.length);

    serverSocket.receive(receivePacket); // Receive UDP packet from client


    String message = new String(receivePacket.getData(), 0, receivePacket.getLength());

    System.out.println("Received from client: " + message);


    if (message.equalsIgnoreCase("exit")) {

        System.out.println("Server exiting...");

        break;

    }

}

serverSocket.close();

}
```

UDP Client (sends messages)

```
import java.net.DatagramPacket;
```

```
import java.net.DatagramSocket;

import java.net.InetAddress;

import java.util.Scanner;


public class UDPClient {

    public static void main(String[] args) throws Exception {

        DatagramSocket clientSocket = new DatagramSocket();

        InetAddress serverAddress = InetAddress.getByName("localhost"); // Server IP address
        (localhost here)

        Scanner scanner = new Scanner(System.in);


        System.out.println("Enter messages to send to server. Type 'exit' to quit.");


        while (true) {

            System.out.print("You: ");

            String message = scanner.nextLine();

            byte[] sendBuffer = message.getBytes();


            DatagramPacket sendPacket = new DatagramPacket(sendBuffer, sendBuffer.length,
serverAddress, 9876);

            clientSocket.send(sendPacket); // Send packet to server
```

```
        if (message.equalsIgnoreCase("exit")) {  
            System.out.println("Client exiting...");  
            break;  
        }  
    }  
    clientSocket.close();  
    scanner.close();  
}  
}
```

How it works:

- Server creates a DatagramSocket on port 9876 and waits for packets.
 - Client creates a DatagramSocket and sends messages to the server's IP and port.
 - The server prints received messages.
 - Both exit when client sends "exit".
-

UDP SOCKET OPTIONS:

◆ What is a UDP Socket Option?

In Java, a **UDP socket** is represented by the `DatagramSocket` class. Like TCP sockets, UDP sockets also have **configurable options** that affect how the socket behaves.

These options can include:

Socket Option	Meaning	
<code>.setSoTimeout(int time)</code>	Sets a timeout for how long <code>receive()</code> waits before giving up. (in milliseconds)	
<code>.setReuseAddress(boolean)</code>	Allows other sockets to bind to the same port. Useful for multicast or quick restart.	
<code>.setBroadcast(boolean)</code>	Enables sending broadcast packets (to all devices in a subnet).	
<code>.setReceiveBufferSize(int)</code>	Sets the buffer size for incoming packets.	
<code>.setSendBufferSize(int)</code>	Sets the buffer size for outgoing packets.	

These options help **tune** the socket's performance or  behavior in different networking conditions.

```
import java.net.DatagramPacket;
```

```
import java.net.DatagramSocket;
```

```
import java.net.InetAddress;
```

```
import java.net.SocketException;
```

```
public class UDPSocketOptionsDemo {
```

```
    public static void main(String[] args) {
```

```
final int PORT = 9876;

try {

    // Create the socket

    DatagramSocket socket = new DatagramSocket(PORT);

    // --- Set UDP Socket Options ---

    socket.setSoTimeout(5000); // Timeout for receive (in milliseconds)

    socket.setReuseAddress(true); // Allow reuse of address/port

    socket.setBroadcast(true); // Enable broadcasting

    socket.setReceiveBufferSize(2048); // Incoming buffer size

    socket.setSendBufferSize(2048); // Outgoing buffer size


    // Display set values for verification

    System.out.println("Receive Buffer Size: " + socket.getReceiveBufferSize());

    System.out.println("Send Buffer Size  : " + socket.getSendBufferSize());

    System.out.println("Broadcast Enabled : " + socket.getBroadcast());

    System.out.println("Reuse Address    : " + socket.getReuseAddress());

    System.out.println("Socket Timeout   : " + socket.getSoTimeout());
```

```
// Prepare buffer to receive data

byte[] receiveBuffer = new byte[2048];

DatagramPacket receivePacket = new DatagramPacket(receiveBuffer,
receiveBuffer.length);

System.out.println("\nServer is waiting for data...");

// Try to receive (with timeout)

try {

    socket.receive(receivePacket);

    String clientMessage = new String(receivePacket.getData(), 0,
receivePacket.getLength());

    System.out.println("Received from client: " + clientMessage);

// Send a reply back

String reply = "Message received: " + clientMessage;

byte[] sendBuffer = reply.getBytes();

InetAddress clientAddress = receivePacket.getAddress();

int clientPort = receivePacket.getPort();
```

```
DatagramPacket    sendPacket    =    new    DatagramPacket(sendBuffer,
sendBuffer.length, clientAddress, clientPort);

    socket.send(sendPacket);

    System.out.println("Reply sent to client.");

} catch (java.net.SocketTimeoutException e) {

    System.out.println("No data received within timeout.");

}

socket.close();

System.out.println("Server socket closed.");

} catch (SocketException e) {

    System.out.println("SocketException: " + e.getMessage());

} catch (Exception e) {

    System.out.println("Exception: " + e.getMessage());

}

}

}
```

◆ UDP Port Scanning Program (from 1024 up)

This program scans local ports starting from 1024, checking which UDP ports are **open** or **in use**. Since UDP is connectionless, you can't "connect" to a port to test if it's open. But we can try to **bind** a DatagramSocket to each port and check whether it throws an exception (already in use).

✅ Java Program to Scan Local UDP Ports

```
import java.net.DatagramSocket;

public class UDPPortScanner {

    public static void main(String[] args) {

        int startPort = 1024;

        int endPort = 65535; // you can limit this to e.g. 2000 to make it faster

        System.out.println("Scanning for open/available UDP ports from " + startPort + " to " +
endPort);

        for (int port = startPort; port <= endPort; port++) {

            try {

                DatagramSocket socket = new DatagramSocket(port); // try binding

                socket.close(); // port is available (not in use)

                System.out.println("Port " + port + " is AVAILABLE.");

            } catch (Exception e) {
```

```
// Port is in use or not available

// Uncomment next line if you want to list unavailable ones too

// System.out.println("Port " + port + " is NOT available.");

}

}

System.out.println("UDP Port scanning completed.");

}

}
```

Write UDP client and server socket programs in which the server identifies the number sent by a client is either even or odd and replies to the client accordingly.

✓ UDP Server (checks even/odd and replies)

Saved memory full ⓘ

java

Copy Edit

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;

public class UDPServerEvenOdd {
    public static void main(String[] args) throws Exception {
        DatagramSocket serverSocket = new DatagramSocket(9876);
        byte[] receiveBuffer = new byte[1024];

        System.out.println("UDP Server is running...");

        while (true) {
            DatagramPacket receivePacket = new DatagramPacket(receiveBuffer, receiveBuffer.length);
            serverSocket.receive(receivePacket);

            String receivedData = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Received from client: " + receivedData);

            String reply;

            try {
                int number = Integer.parseInt(receivedData.trim());
                reply = (number % 2 == 0) ? "Even Number" : "Odd Number";
            } catch (NumberFormatException e) {
                reply = "Invalid number!";
            }

            byte[] sendBuffer = reply.getBytes();
            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();

            DatagramPacket sendPacket = new DatagramPacket(sendBuffer, sendBuffer.length, clientAddress, clientPort);
            serverSocket.send(sendPacket);
        }
    }
}
```

✅ UDP Client (sends number, receives even/odd response)

```
java                                                                    Copy Edit

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.util.Scanner;

public class UDPClientEvenOdd {
    public static void main(String[] args) throws Exception {
        DatagramSocket clientSocket = new DatagramSocket();
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number to send to the server: ");
        String number = scanner.nextLine();
        byte[] sendBuffer = number.getBytes();

        InetAddress serverAddress = InetAddress.getByName("localhost");
        int serverPort = 9876;

        DatagramPacket sendPacket = new DatagramPacket(sendBuffer, sendBuffer.length, serverAddress);
        clientSocket.send(sendPacket);

        // Receive reply
        byte[] receiveBuffer = new byte[1024];
        DatagramPacket receivePacket = new DatagramPacket(receiveBuffer, receiveBuffer.length);
        clientSocket.receive(receivePacket);

        String reply = new String(receivePacket.getData(), 0, receivePacket.getLength());
        System.out.println("Server says: " + reply);

        clientSocket.close();
        scanner.close();
    }
}
```

Write a program to create a UDP server and client which converts the date from the client in BS to AD format. [Assume that the difference between BS and AD is 56 years, 08 months, 17 days]

```
import java.net.*;
import java.io.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.Period;

public class DateConverterUDPServer {
    public static void main(String[] args) throws Exception {
        DatagramSocket serverSocket = new DatagramSocket(9876);
        byte[] receiveData = new byte[1024];
        byte[] sendData;

        System.out.println("UDP Server is running...");

        while (true) {
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);

            String bsDateStr = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Received BS Date: " + bsDateStr);

            // Convert BS to AD (assumed Logic)
            DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");
            LocalDate bsDate = LocalDate.parse(bsDateStr, formatter);

            // Subtract 56 years, 8 months, 17 days
            Period diff = Period.of(56, 8, 17);
            LocalDate adDate = bsDate.minus(diff);

            String adDateStr = adDate.format(formatter);

            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();
            sendData = adDateStr.getBytes();

            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, clientAddress);
            serverSocket.send(sendPacket);
            System.out.println("Sent AD Date: " + adDateStr);
        }
    }
}
```

Sure, Ureka! Here's a simple Java program that demonstrates **Multicast communication** using MulticastSocket. It includes:

- A **Sender** that sends a message to a multicast group.
 - A **Receiver** that joins the group and receives that message.
-

Multicast Sender Program

```
import java.net.DatagramPacket;

import java.net.InetAddress;

import java.net.MulticastSocket;


public class MulticastSender {

    public static void main(String[] args) {

        try {

            // Multicast group IP and port

            InetAddress group = InetAddress.getByName("230.0.0.1");

            int port = 4446;


            // Create message

            String message = "Hello, Multicast Receivers!";
```

```
byte[] buffer = message.getBytes();

// Create multicast socket

MulticastSocket socket = new MulticastSocket();

DatagramPacket packet = new DatagramPacket(buffer, buffer.length,
group, port);

// Send packet

socket.send(packet);

System.out.println("Message sent: " + message);

socket.close();

} catch (Exception e) {

    e.printStackTrace();

}

}

}
```

```
import java.net.DatagramPacket;

import java.net.InetAddress;

import java.net.MulticastSocket;


public class MulticastReceiver {

    public static void main(String[] args) {

        try {

            // Multicast group IP and port

            InetAddress group = InetAddress.getByName("230.0.0.1");

            int port = 4446;


            // Create multicast socket and bind to port

            MulticastSocket socket = new MulticastSocket(port);


            // Join group

            socket.joinGroup(group);

            System.out.println("Joined multicast group. Waiting for message...");


            // Prepare buffer to receive data
```

```
byte[] buffer = new byte[1024];

DatagramPacket packet = new DatagramPacket(buffer, buffer.length);


// Receive packet

socket.receive(packet);

String message = new String(packet.getData(), 0, packet.getLength());


System.out.println("Received message: " + message);


// Leave group and close socket

socket.leaveGroup(group);

socket.close();

} catch (Exception e) {

    e.printStackTrace();

}

}

}
```

 Note:

- Run the **Receiver first**, then run the **Sender**.
 - Ensure both programs use the **same port and multicast address**.
 - Multicast might not work properly without network support (especially in virtual/host-only adapters).
-

NON-BLOCKING IO: